

Séance 1 — Machine, logiciel et bases de Python

Introduction à la programmation Python — IUT 1re année

Support de cours

Séance 1

Machine, logiciel et bases de Python

Architecture matérielle

Un ordinateur est composé de plusieurs éléments qui collaborent :

- **CPU** (processeur) : exécute les instructions, cadencé en GHz
- **RAM** : mémoire de travail, rapide mais **volatile** (perdue à l'extinction)
- **Stockage** (disque dur / SSD) : lent mais **persistant**
- **Périphériques** : clavier, écran, capteurs, ports de communication (USB, série...)

Exemple concret : quand un programme calcule une moyenne sur un gros fichier, les données sont chargées du disque vers la RAM, puis traitées par le CPU.

Architecture logicielle

- **Système d'exploitation (OS)** : Windows, Linux, macOS — gère les ressources matérielles et exécute les programmes
- **Programme compilé** (C, C++) : traduit intégralement en langage machine *avant* exécution → rapide mais dépend de la plateforme
- **Programme interprété** (Python) : traduit et exécuté ligne par ligne par un **interpréteur** → portable, plus lent

Exemple : un même script Python (`.py`) s'exécute sans modification sur Windows, Linux ou macOS, tant que Python est installé.

Installer et lancer Python

1. **Télécharger** Python depuis python.org (cocher "Add to PATH" sous Windows)

2. **Vérifier l'installation** :

```
python --version
```

3. **Lancer la console interactive** (idéale pour tester une ligne rapidement) :

```
python >>> 2 + 2 4
```

4. Créer un script : fichier texte `mon_script.py`

```
print("Bonjour")
```

5. Lancer le script :

```
python mon_script.py
```

Variables et types de données

Une **variable** est un nom qui désigne une valeur stockée en mémoire.

```
age = 20 # int -> nombre entier temperature = 20.5 # float -> nombre à virgule nom  
= "Alice" # str -> chaîne de caractères actif = True # bool -> booléen (True /  
False)
```

Python déduit le type automatiquement (**typage dynamique**), mais chaque type a ses propres règles de calcul.

Conversions explicites

On peut convertir explicitement une valeur d'un type vers un autre :

```
x = "42" y = int(x) + 1 # "42" -> 42 (int), puis + 1 = 43 z = str(3.14) # 3.14 ->  
"3.14" (str) w = float("3,14".replace(",", ".")) # gérer une virgule décimale
```

■ ■ Certaines conversions échouent : `int("bonjour")` lève une erreur.

Les opérateurs

Arithmétiques

```
7 + 2 # addition -> 9 7 - 2 # soustraction -> 5 7 * 2 # multiplication -> 14 7 / 2  
# division réelle -> 3.5 7 // 2 # division entière -> 3 7 % 2 # modulo (reste) ->  
1 7 ** 2 # puissance -> 49
```

Affectation combinée

```
x = 5 x += 1 # équivalent à x = x + 1 x *= 2
```

Comparaison

```
a == b # égalité a != b # différence a < b, a <= b, a > b, a >= b
```

Le type influence le résultat !

```
>>> 7 // 2 3 >>> 7 / 2 3.5 >>> 0.1 + 0.2 0.30000000000000004
```

Les flottants ont une précision **limitée** (norme IEEE 754) : en métrologie, ne jamais comparer deux flottants avec `==`, préférer une tolérance.

Entrées / sorties simples

```
valeur = input("Température en °C : ") # input() renvoie toujours une str
temperature = float(valeur) # conversion nécessaire print(f"Température :
{temperature} °C") # f-string : insère une variable dans un texte print(f"Arrondi
: {temperature:.1f} °C") # formatage : 1 chiffre après la virgule
```

Tests logiques

```
if temperature < 0: print("Gel") elif temperature < 30: print("Normal") else:
print("Chaud")
```

Opérateurs logiques pour combiner des conditions :

```
if temperature > 0 and temperature < 30: print("Plage normale") if not actif or
temperature > 100: print("Arrêt d'urgence")
```

À vous de jouer

Direction les exercices de la séance 1 →

Exercices

Exercices — Séance 1 : Machine, logiciel et bases de Python

Ex. 1 — Premier script Créer un fichier `bonjour.py` qui affiche votre nom et votre filière, puis l'exécuter depuis un terminal.

Ex. 2 — Calculatrice d'opérandes Demander deux nombres à l'utilisateur (`input`) et afficher le résultat de `+` `-` `*` `/` `//` `%` `**` entre eux, avec des f-strings lisibles.

Ex. 3 — Conversions de types Demander une chaîne comme `"3,14"` (virgule), la convertir en `float` utilisable, puis en `str` formatée à 2 décimales.

Ex. 4 — Précision flottante Calculer `0.1 + 0.2`, comparer à `0.3` avec `==` puis avec une tolérance (`abs(a-b) < 1e-9`). Expliquer la différence en commentaire.

Ex. 5 — Classification d'une mesure Demander une pression (bar) et afficher "normale", "alerte" ou "danger" selon des seuils donnés (`if / elif / else`, `and/or`).

Ex. 6 — Calcul d'IMC Demander poids (kg) et taille (m), calculer l'IMC, l'afficher formaté à 1 décimale, puis afficher une catégorie (maigreur/normal/surpoids) selon des seuils.